Evolutionary Multi-Objective Deep Reinforcement Learning for Task Offloading in Industrial Internet of Things

Xu Liu, Zheng-Yi Chai, Yan-Yang Cheng, Ya-Lun Li, and Tao Li

***Abstract*—Mobile Edge Computing (MEC) plays a pivotal role in optimizing the Industrial Internet of Things (IIoT), where the Industrial Task Offloading Problem (ITOP) is crucial for ensuring optimal system performance by balancing conflicting objectives such as delay, energy consumption, and cost. However, existing approaches often oversimplify multi-objective optimization by aggregating conflicting goals into a single objective, while also suffering from limited exploration and robustness in uncertain MEC scenarios within IIoT. To overcome this limitation, we propose EMDRL-ITOP, an Evolutionary Multi-Objective Deep Reinforcement Learning algorithm that synergizes evolutionary algorithm with deep reinforcement learning (DRL). Firstly, we formulate a multi-objective task scheduling model for IIoT-MEC and design a three-dimensional vector reward function within a Multi-Objective Markov Decision Process framework, enabling simultaneous optimization of delay, energy, and cost. Then, EMDRL-ITOP integrates evolutionary mechanisms to enhance exploration and robustness: a dynamic elite selection strategy prioritizes high-quality policies, a distillation crossover operator fuses advantageous traits from elite strategies, and a proximal mutation mechanism maintains population diversity. These components collectively improve learning efficiency and solution quality in dynamic environments. Extensive simulations across six instances demonstrate that EMDRL-ITOP achieves a superior balance among conflicting objectives compared to state-of-the-art methods, while also outperforming existing algorithms in several key performance metrics.**

***Index Terms*—Industrial Internet of Things (IIoT), mobile edge computing (MEC), task offloading, evolutionary algorithm, multi-objective optimization, deep reinforcement learning.**

Received 25 May 2024; revised 12 March 2025; accepted 24 June 2025. Date of publication 2 July 2025; date of current version 7 October 2025. This work is partially supported by the National Natural Science Foundation of China under Grant 62272248,62172298, the Natural Science Foundation of Tianjin under Grant 23JCZDJC01010, 23JCQNJC00010, 22JCZDJC00600, the Key Program of Science and Technology Foundation of Tianjin under Grant 4HHXCSS00004, the Innovation Fund for Combination of industry and education in China under Grant 2022BL083. The associate editor coordinating the review of this article and approving it for publication was P. Varga. *(Xu Liu and Zheng-Yi Chai contributed equally to this work.) (Corresponding author: Tao Li.)*

Xu Liu is with the College of Computer Science and the Tianjin Key Laboratory of Network and Data Science Technology, Nankai University, Tianjin 300071, China (e-mail: liu.x@mail.nankai.edu.cn).

Zheng-Yi Chai is with the School of Computer Science and Technology and the Tianjin Key Laboratory of Autonomous Intelligence Technology and Systems, Tiangong University, Tianjin 300387, China (e-mail: chaizhengyi@tiangong.edu.cn).

Yan-Yang Cheng is with the School of the Computer College, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: 2024010310@bupt.cn).

Ya-Lun Li is with the School of Electronic and Information Engineering, Tiangong University, Tianjin 300387, China (e-mail: liyalun@tiangong.edu.cn).

Tao Li is with the College of Computer Science and the Haihe Lab of ITAI, Nankai University, Tianjin 300071, China (e-mail: litao@nankai.edu.cn).

Digital Object Identifier 10.1109/TNSM.2025.3585148

I. Introduction

With the development of Industry 4.0 and big data, the Industrial Internet of Things (IIoT) has become increasingly important in intelligent manufacturing, enabling real-time information exchange and fast data processing [1]. The traditional way of industrial data processing is through cloud computing, where Smart Factory Equipment (SFE) uploads industrial data to cloud servers for processing. While cloud servers possess abundant computational capabilities, the exponential growth of industrial data presents challenges in network bandwidth utilization. Furthermore, the geographical separation between cloud servers and SFE leads to significant transmission delays. Cloud computing can affect the Quality-of-Service (QoS) of industrial applications in IIoT [2].

To address these challenges, researchers have proposed a new computing model called Mobile Edge Computing (MEC) [3]. By deploying edge servers with sufficient computing resources at the network edge, MEC can effectively solve network congestion and high delay [4]. In MEC of IIoT (IIoT-MEC), industrial tasks can be executed on SFE or MEC servers, which is known as the Industrial Task Offloading Problem (ITOP). Executing tasks on SFE can reduce task execution delay but leads to higher energy consumption, while executing tasks on MEC servers can reduce device energy consumption but increases system delay and cost [5]. Thus, making a reasonable offloading decision is essential to meet the QoS requirements of IIoT applications.

For the ITOP in IIoT-MEC systems, existing research can be mainly classified into two categories: traditional optimization methods and Deep Reinforcement Learning (DRL) methods [6]. Traditional optimization methods include convex optimization algorithms, Evolutionary Algorithms (EAs), and game theory, etc, which can achieve good performance in static environments. However, the IIoT-MEC environment is highly complex dynamic, and full of uncertainties [7]. Therefore, when using traditional algorithms to solve the ITOP, will cause a waste of computing resources and fail to obtain reasonable offloading solutions.

DRL has significant advantages in handling complex environments, as they can interact with the fast-changing environment [8], [9], [10], [11]. When solving ITOP, system

1932-4537 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:55 UTC from IEEE Xplore. Restrictions apply.

delay, energy consumption, and cost are usually the three important indicators of performance [12], where cost refers to payment for using the computing resources in the MEC server.However, existing DRL-based algorithms for solving multi-objective problems like ITOP often convert multiple objectives into a single objective by weighted sum [13]. This way ignores the conflicts among multiple objectives and it is difficult to set weights for each objective. For instance, when dealing with delay-sensitive applications, users tend to prioritize reducing system delay. When the SFE energy is insufficient, users prefer to reduce energy consumption. Additionally, when the user's budget is limited, they tend to reduce the cost of MEC [14]. Moreover, existing works mainly focus on single-policy optimization methods, which can only obtain one offloading solution and have poor exploration capability to the environment and low robustness [15].

To address the above-mentioned challenges, we propose an Evolutionary Multi-objective Deep Reinforcement Learning (EMDRL) algorithm for ITOP in IIoT-MEC, leveraging the complementary strengths of EAs and DRL. EAs excel in global search and diversity maintenance, making them well-suited for exploring the Pareto front in multi-objective optimization problems. In contrast, DRL is highly effective in dynamic decision-making by continuously learning from the environment. However, traditional DRL approaches often suffer from poor exploration, leading to suboptimal solutions, while EA-based approaches lack adaptability in dynamic environments. By integrating EA's global search ability with DRL's adaptive learning capability, EMDRL can efficiently explore the solution space, improve convergence, and generate a diverse set of high-quality offloading solutions close to the Pareto front. To the best of our knowledge, this is the first work that applies an EMDRL-based approach to ITOP in IIoT-MEC, providing an effective trade-off between delay, energy consumption, and cost.

The main contributions of this paper can be summarized as follows:

- We have constructed a multi-objective task scheduling model for the IIoT-MEC, considering the conflicting objectives of delay, energy consumption, and cost. Additionally, we also proposed a multi-objective Markov decision process (MOMDP) framework based on a three-dimensional vector reward function, which effectively addresses the limitations of traditional DRL in single-objective optimization.
- To address the ITOP, we propose EMDRL-ITOP, which synergizes the global search capability of evolutionary algorithms with the adaptive decision-making of DRL. EMDRL-ITOP introduces three key mechanisms: dynamic elite selection strategy, distillation crossover operator, and proximal mutation mechanism, effective generation of non-dominated solutions near the Pareto front that dynamically adapt to diverse requirements.
- We conducted extensive simulation experiments on six instances. The results demonstrate that EMDRL-ITOP can effectively balance multiple objectives and significantly outperforms existing state-of-the-art algorithms in several key performance metrics.

The remaining structure of this paper is as follows. In Section II, the related work and the research motivation of this paper are analyzed. In Section III, the system model is established and the problem is described. In Section IV, we provide a detailed description of the proposed algorithm. Section V presents experimental results and analyzes the results. Section VI concludes the paper and proposes future research directions.

II. Related Work and Research Motivation

A. Problem Background

The ITOP in IIoT-MEC systems aims to optimize the allocation of computationally intensive tasks between resource-constrained SFE and edge servers. Tasks generated by SFE can either be processed locally or offloaded to edge servers through wireless links. This decision-making process mainly considers balancing three objectives:

1) **Delay**: Excessive delay can affect system stability and production efficiency. Therefore, it is necessary to ensure that tasks are completed in the shortest possible time to meet the low-delay requirements of industrial applications.
2) **Energy consumption**: Optimize the energy consumption of task execution to extend the lifespan of devices and reduce maintenance costs.
3) **Costs**: Reduce the computational cost associated with offloading tasks to edge servers, thereby making rational use of edge computing resources and improving economic efficiency.

However, these three objectives conflict with each other. For example, reducing delay typically requires increasing the utilization of computing resources, which in turn increases energy consumption. To reduce energy consumption, SFE may offload more tasks to the edge, but this can lead to higher computational costs. Moreover, while offloading tasks can reduce local computation delay, frequent use of edge server resources also increases operational expenditure. Therefore, finding a balance between delay, energy consumption, and computational cost is the core challenge of ITOP in IIoT-MEC. Existing solutions for ITOP can be broadly classified into traditional algorithms and DRL algorithms. Below, we provide a detailed analysis of each approach.

B. Research on Solving ITOP by Traditional Algorithms

Traditional algorithms for solving ITOP include EAs, game theory, convex optimization algorithms, etc. In IIoT-MEC, when dealing with delay-sensitive tasks, the system's delay is prioritized optimizing to meet users' needs. Bebortta et al. [16] proposed a consolidated stochastic computation offloading framework to address the increasing computational requirements in IIoT-MEC systems, which reduced the system delay. Dai et al. [17] presented a layered network architecture and adopted an improved marine predators algorithm to solve ITOP. Xue et al. [18] proposed an optimization method based on access points' cooperation and reduced delay by jointly allocating computing and transmission resources.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:55 UTC from IEEE Xplore. Restrictions apply.

When the device's energy is insufficient or when handling large tasks, optimizing energy consumption becomes particularly important. Hsu et al. [19] suggested an offloading algorithm to jointly optimize the allocation of computing resources with the minimum energy consumption. Wu et al. [20] designed an energy-efficient task offloading strategy, and adopted the Conditional Value-at-Risk framework to find the optimal offloading solution. Jian et al. [21] proposed an improved differential evolution algorithm and sequential quadratic programming algorithm to solve the ITOP, which reduced system energy consumption.

In the IIoT-MEC environment, there are multiple conflicting objectives, such as delay, energy consumption, MEC cost, and system security. In general, users prefer to achieve a balance among multiple objectives. Therefore, how to optimize multiple objectives simultaneously is one of the current research hotspots. Peng et al. [22] regarded ITOP as a multi-objective problem, and proposed an improved NSGA-III algorithm, which reduced system delay and energy consumption at the same time. Zhang et al. [23] transformed ITOP into a Mixed Integer Nonlinear Programming problem, and suggested a novel task offloading scheme for optimizing the delay and energy consumption. Jin et al. [24] proposed an improved differential evolution algorithm, and jointly optimized offloading decisions and resource allocation. Zhou et al. [25] investigated the offloading problem by minimizing the delay and energy consumption, and proposed an improved fast non-dominated sorting algorithm to solve ITOP. Chai et al. [26] presented an improved decomposition-based multi-objective evolutionary algorithm to minimize delay and energy consumption. Peng et al. [27] proposed a decomposition-based constrained multi-objective evolutionary algorithm, which utilizes infeasible individuals with large objective values to provide valuable information, thereby reducing energy consumption and ensuring safety.

Qiu et al. [28] summarized the existing works on ITOP using traditional algorithms and pointed out their limitations and shortcomings. Although traditional algorithms have achieved good results in solving ITOP in some static environments, IIoT-MEC is a complex and dynamic environment full of uncertainties. In this case, the performance of traditional algorithms in solving ITOP is not ideal. DRL-based algorithms can interact with the IIoT-MEC environment and have powerful data processing capabilities. Therefore, more and more researchers are adopting DRL-based algorithms to solve ITOP.

C. Research on Solving ITOP by DRL Algorithms

Unlike traditional algorithms, DRL-based algorithms have gained attention because they can actively interact with and learn from the environment, allowing them to adapt to dynamic and uncertain conditions in IIoT-MEC systems.

Wang et al. [29] presented three DRL-based frameworks to address the channel allocation and task offloading problem in IIoT-MEC systems, reducing transmission delay and device energy consumption. To maximize the long-term computing performance, Chen et al. [30] presented a method based on long-term and short-term memory and DRL technology. Chen et al. [31] maximized computation rate by jointly optimizing wireless power transfer duration and offloaded energy consumption per task and proposed a DRL-based offloading method to quickly get offloading solutions. Zhao et al. [32] formulated the joint optimization problem of security, task offloading, and resource allocation strategies, and presented a real-time, safe, and efficient offloading method based on DRL. Song et al. [33] proposed a multi-objective DRL algorithm by dynamic weight adjustment to reduce system delay, energy consumption, and cost. Yi et al. [34] presented a DRL-driven intelligent joint optimization strategy to solve the joint optimization of computation offloading and resource allocation, reducing the total cost of the system and improving the success rate of task processing.

However, existing works only focus on single-policy DRL algorithms, and aggregate multiple objectives into a single objective by weighted sum for training optimization and obtaining a task offloading solution. The above-mentioned algorithms ignore the conflicts between multiple objectives and cannot meet the system's requirements for multiple objectives. The evolutionary deep reinforcement learning (EDRL) algorithm is a multi-policy DRL algorithm that combines evolutionary optimization and DRL to obtain a set of offloading solutions. Khadka and Tumer [35] first combined EA with the DRL algorithm and proposed the EDRL algorithm, which achieved good results on multiple test problems. Pourchot and Sigaud [36] presented an improved EDRL algorithm, which performed better. They established a communication network to enhance service quality and achieve optimal allocation within the MEC environment. Song et al. [37] applied EDRL to the task offloading problem in UAV-assisted MEC systems to reduce delay and energy consumption. It can be seen that the EDRL algorithm has achieved good results in some areas, but there is no work on its application to IIoT-MEC systems yet.

D. Research Motivation

The Industrial Task Offloading Problem (ITOP) in IIoT-MEC systems is critical for balancing delay, energy efficiency, and reliability in dynamic industrial environments. While DRL has shown promise in addressing ITOP, existing approaches suffer from two fundamental limitations:

1) **Ignoring Objective Conflicts**: Existing DRL methods use the weighted sum to merge conflicting objectives (e.g., delay and energy) into a single reward, oversimplifying multi-objective optimization and failing to achieve Pareto-optimal trade-offs.
2) **Limited Exploration and Robustness**: Existing DRL methods often have limited ability to explore the environment and lack robustness in dynamic and uncertain conditions.

To address the aforementioned issues, we propose EMDRL-ITOP by integrating Evolutionary Algorithms (EAs) and DRL. Firstly, we design a three-dimensional vector reward function to directly model the dynamic trade-offs between delay, energy consumption, and cost within the MOMDP framework.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:55 UTC from IEEE Xplore. Restrictions apply.

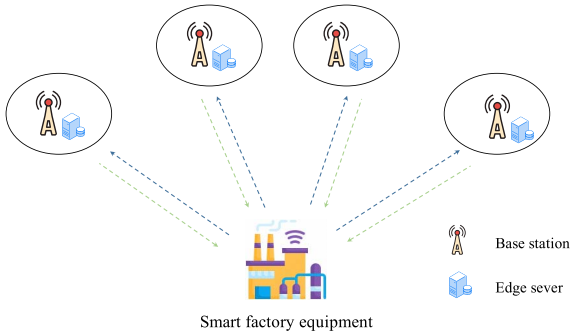


Fig. 1. Architecture of IIoT-MEC system. It includes the SFE, edge servers, base stations, and transmission links, illustrating that tasks can either be executed locally on the SFE or offloaded to an edge server.

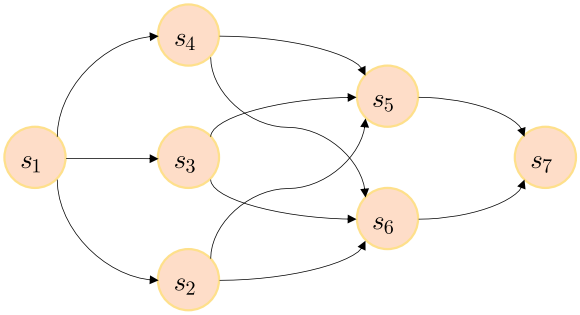


Fig. 2. Simple DAG example illustrates the dependency relationships among industrial tasks. Nodes represent individual tasks, while edges denote the precedence constraints between them, effectively highlighting the dependencies in task scheduling.

Additionally, we introduce a dynamic elite selection strategy to filter out high-quality policies, employ distillation crossover operations to integrate the advantages of elite policies and design a proximal mutation mechanism to enhance population diversity. This allows for rapid convergence to a high-quality set of solutions in dynamic and uncertain environments. EMDRL-ITOP combines the global search capability of EAs with the real-time decision-making advantage of DRL, yielding a set of non-dominated solutions close to the Pareto front, and users can dynamically select the optimal strategy based on their requirements.

III. System Modeling and Problem Description

In this paper, we establish an IIoT-MEC system. Similar to the system models proposed in [33], the IIoT-MEC system includes an SFE and $\mathcal{F} = \{1, \ldots, F\}$ MEC base stations, each equipped with a MEC server, as shown in Fig. 1. There are N computing-intensive industrial tasks in the SFE, each of which can be executed on the SFE or offloaded to any MEC server. In addition, it is assumed that only one task can be performed per time slot t [38], [39], $t = 1, \ldots, N$.

The Directed Acyclic Graph (DAG) is a popular task scheduling method, it can be used to calculate the task execution order [38]. Therefore, to improve task execution efficiency, the industrial tasks are modeled as a DAG $A = (S, C)$. Where S and C represent the task set and task dependency constraints, as shown in Fig. 2. Denote the set of direct predecessors of task s_i as $DP(s_i)$ and the set of direct successors as $DS(s_i)$, where $i \in \{1, \ldots, N\}$. $C(s_i, s_j)$ represents the dependency constraint from task s_i to s_j $(i \neq j)$, which means s_j must be completed after s_i is completed. Here, $s_j \in DS(s_i)$ represents the direct successor of s_i, and correspondingly, $s_i \in DP(s_j)$ represents the direct predecessor of s_j. If $DS(s_i)$ is empty, s_i is the end task in the task set. Conversely, if $DP(s_i)$ is empty, s_i is the start task. In particular, the DAG in this work is designed offline based on the industrial task dependency. The DAG is processed as a fixed structure upon request, with no modifications to its topology or dependencies during runtime, ensuring reliability [40]. In Fig. 2, s_1 and s_7 represent the start and end tasks of the task set, respectively. Table I shows the main symbols used in this paper.

TABLE I
Main Symbol (1)

Symbol	Definition
F	Number of MEC servers
$\mathcal{F}$	Set of MEC servers
N	Number of tasks in SFE
A	Industrial application in SFE
S	Task set in SFE
C	Task dependency constraints in SFE
s_i	The i-th task in SFE
c_i	Number of CPU cycles required to execute s_i
$DP(s_i)$	Set of direct predecessors of task s_i
$DS(s_i)$	Set of direct successors of task s_i
D_i^l	Time delay of task s_i execution in SFE
E_i^l	Energy consumption of s_i execution in SFE
D_i^s	Time delay of transmitting input data of task s_i to m_f
E_i^s	Energy consumption of transmitting input data of task s_i to m_f
D_i^z	Time delay of task s_i execution in m_f
D_i^r	Time delay of SFE receiving output data of s_i
E_i^r	Energy consumption of SFE receiving output data of task s_i
cp_l	Computing power of SFE
m_f	The f-th edge server
cp_{m_f}	Computing power of the f-th edge server
ids_i	Input data size of task s_i
ods_i	Output data size of task s_i
p_s	Transmission power of SFE
p_r	Reception power of SFE
O	Task execution order in SFE
R_{m_f}	Transmission rate of wireless channel from SFE to m_f

A. Local Computing Model

In this paper, when an industrial task s_i is executed locally, the completion delay of s_i in SFE is denoted as CD_i^l. When the task is offloaded to an edge server for execution, let CD_i^s denote the completion delay when the input data of s_i are completely offloaded to edge server m_f, where $f = \{1, \ldots, F\}$. Let CD_i^z denote the completion delay of s_i execution on m_f, and CD_i^r denote the completion delay of SFE receiving output data of s_i from edge server m_f. If $CD_i^s = CD_i^z = CD_i^r = 0$, it means that s_i is executed locally. Otherwise, when s_i is executed on an edge server, $CD_i^l = 0$.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:55 UTC from IEEE Xplore. Restrictions apply.

During the task execution process, the execution order of tasks is considered, i.e., the current task s_i can only be executed after its direct predecessor tasks have been completed. Therefore, when s_i is executed on SFE, its start delay SD_i^l is defined as:

$$SD_i^l = \max\{\max\{CD_j^l, CD_j^r\} | s_j \in DP(s_i)\} \tag{1}$$

When s_i is executed on SFE, its delay and energy consumption are defined as follows:

$$D_i^l = \frac{c_i}{cp_l} \tag{2}$$

$$E_i^l = h \cdot c_i \cdot (cp_l)^2 \tag{3}$$

where c_i represents the number of CPU cycles required to execute s_i, cp_l represents the computing power of SFE. And h is the effective capacitance coefficient related to the chip.

B. IIoT-MEC Computing Model

When s_i is executed on edge server m_f, the delay and energy consumption of transmitting s_i to m_f are defined as follows:

$$D_i^s = \frac{ids_i}{R_{m_f}} \tag{4}$$

$$E_i^s = p^s \cdot D_i^s \tag{5}$$

where ids_i represents the size of input data of s_i, and p_s represents the transmission power of SFE. According to Shannon's theorem [41], the transmission rate R_{m_f} of wireless channel from SFE to m_f is defined as:

$$R_{m_f} = B \cdot log_2\left(1 + \frac{p_s \cdot G_{m_f}}{\sigma^2}\right) \tag{6}$$

where B and G_{m_f} represent the channel bandwidth and channel gain between SFE and m_f, respectively, and σ^2 represents the noise power.

It is worth noting that the start delay of transmitting s_i must be after the completion of its direct predecessor tasks. When $s_j \in DP(s_i)$ is executed on SFE, the start delay of s_i is $SD_i^s = CD_j^l$. When s_j and s_i are offloaded to the same edge server for execution, $SD_i^s = CD_j^s$. Otherwise, when they are offloaded to different edge servers for execution, $SD_i^s = CD_j^r$. s_i must be executed on m_f after the transmission is completed, i.e., its start delay is $SD_i^z = CD_i^s$. In addition, when s_j and s_i are executed on the same edge server, $SD_i^z = CD_j^z$. Since IIoT-MEC servers have sufficient computing resources, the energy consumption of s_i is ignored when it is executed on m_f. When it is executed on m_f, the delay of s_i is defined as follows:

$$D_i^z = \frac{c_i}{cp_{m_f}} \tag{7}$$

where cp_{m_f} represents the computing power of m_f. After s_i is completed, m_f starts to return output data of s_i to SFE. Therefore, its start delay is $SD_i^r = SD_i^z$. The delay and energy consumption of receiving output data of s_i to SFE are defined as follows:

$$D_i^r = \frac{ods_i}{R_{m_f}} \tag{8}$$

$$E_i^r = p^r \cdot D_i^r \tag{9}$$

where ods_i represents the size of output data of s_i, and p^r represents the reception power of SFE.

In summary, the total delay of s_i offloaded to execute on m_f is $D_i^m = D_i^s + D_i^z + D_i^r$, and the total energy consumption is $E_i^m = E_i^s + E_i^r$.

In addition, when tasks are offloaded to MEC servers, users need to pay for the cost of computing resources in the MEC server to the MEC system operator. The cost is related to the computing power of the server and is defined as follows:

$$COST\left(cp_{m_f}\right) = e^{-\alpha} \cdot \left(e^{cp_{m_f}} - 1\right) \cdot \beta \tag{10}$$

where α and β are two coefficients. The cost of s_i when it is executed on m_f is defined as follows:

$$COST_i^m = COST\left(cp_{m_f}\right) \cdot D_i^z \tag{11}$$

C. Problem Description

The ITOP of IIoT-MEC proposed in this paper has multiple conflicting objectives, including delay, energy consumption, and cost. For example, allocating different tasks to various MEC servers for execution can reduce SFE's energy consumption but requires more MEC costs and delays. This paper aims to simultaneously optimize these three objectives to solve the ITOP and meet user requirements.

For an application A in SFE, which includes multiple tasks, the delay of SFE is defined as:

$$D(A) = \max\{\max\{D_i^l, D_i^r\} | s_i \in last(A)\} \tag{12}$$

where $s_i \in last(A)$ represents the end task in A. $\max\{D_i^l, D_i^r\}$ represents the completion time of s_i. If the task is executed on SFE, it equals D_i^l. Otherwise, it equals D_i^r.

Secondly, the energy consumption in the IIoT-MEC system includes not only the energy consumption generated by SFE, denoted as E_i^l, but also the energy consumption generated by offloading to m_f, denoted as E_i^m. Assumes that $E_i^m = 0$ when s_i is executed on SFE, and $E_i^l = 0$ when s_i is offloaded to m_f. The total energy consumption is defined as follows:

$$E(A) = \sum_{i=1}^{N}\left(E_i^l + E_i^m\right) \tag{13}$$

In the IIoT-MEC system, we assume that the cost for MEC servers is only required when s_i is offloaded to m_f for execution. When s_i is executed on SFE, $COST_i^m = 0$. Therefore, the total cost is defined as:

$$COST(A) = \sum_{i=1}^{N} COST_i^m \tag{14}$$

In addition, to achieve more efficient processing of industrial tasks in the IIoT-MEC system, the task dependency constraints are defined as:

$$\begin{aligned} Min \ \ & (D(A), E(A), COST(A)) \\ & C_1: m_f \in \{1, \ldots, F\}, \forall s_i \in S, i \in \{1, \ldots, N\}, \\ & C_2: CD_j^l \leq SD_i^l, \forall s_i \in S, s_j \in DP(s_i), \end{aligned}$$

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:55 UTC from IEEE Xplore. Restrictions apply.

TABLE II
TASK EXECUTION DELAY (SECONDS). IT INCLUDES THE EXECUTION DELAYS OF DIFFERENT TASKS ON THE SFE AND THE VARIOUS EDGE SERVERS, AS WELL AS THE ACTUAL EXECUTION LOCATION FOR EACH TASK (WHERE 0 INDICATES EXECUTION ON THE SFE AND A NON-ZERO VALUE INDICATES EXECUTION ON THE CORRESPONDING EDGE SERVER)

Task	SFE	MEC server 1	MEC server 2	Execution location
s_1	4	1	1	2
s_2	3	2	1	1
s_3	5	2	1	0
s_4	4	3	2	1
s_5	7	3	2	1
s_6	4	1	1	2
s_7	5	3	2	0

$$C_3: CD_j^r \le SD_i^l, \forall s_i \in S, s_j \in DP(s_i),$$
$$C_4: CD_i^s \le SD_i^z, \forall s_i \in S,$$
$$C_5: CD_j^z \le SD_i^z, \forall s_i \in S, s_j \in DP(s_i). \quad (15)$$

C1 represents the execution location constraint for tasks. Any task s_i can be executed on SFE or edge server m_f. C2 and C3 represent constraints when s_i is executed on SFE, indicating that s_i must be executed only after its direct predecessor task s_j is completed. If s_j is executed on m_f, it must ensure that the output data of s_j has been fully transmitted to SFE. Conversely, if s_j is executed on SFE, it must ensure that s_j is completed. CD_j^l and CD_j^r denote the completion delay of executing s_j at the SFE and the completion delay of receiving the output data of s_j from the edge server, respectively. SD_i^l represents the start delay of executing s_j at the SFE. $DP(s_i)$ denotes the set of direct predecessors of task s_i. C4 and C5 represent constraints when s_i is executed on m_f, where C4 indicates that before executing s_j on m_f, it must ensure that the input data of s_j has been fully transmitted to m_f. C5 represents the constraint when s_i and s_j are offloaded to the same MEC server, indicating that before executing s_j on m_f, it must ensure that s_j has been completed on m_f. CD_i^s denotes the completion delay for offloading the input data of s_i to m_f. CD_j^z denotes the completion delay of executing s_j on m_f. SD_i^z represents the start delay of executing s_j on m_f.

To better understand the task execution process, a simple example is described. Suppose there is an SFE and two MEC servers, m_1 and m_2, in the IIoT-MEC system. There is an application A in SFE, which includes a task set $S = \{s_1, s_2, s_3, s_4, s_5, s_6, s_7\}$. According to Fig. 2, the task execution order $O = \{s_1, s_2, s_4, s_3, s_6, s_5, s_7\}$ is obtained. In this example, to better understand the process of local computing and edge computing, assume that the delay for SFE to transmit s_i, $i = 1, \ldots, 7$, and receive the output data returned by the MEC server is 1 second and the power consumption of executing on SFE is 1 watt. The transmission power p^s and receiving power p^r of SFE are both 0.1 watts, and the cost $COST(cp_{m_f})$ is 0.1\$. Table II shows the execution delay and execution location of s_i, where the starting and ending tasks are s_1 and s_7, respectively. By calculation, the total delay of the task execution process is $D(A) = 21s$, the total energy consumption is $E(A) = 10J$, and the total cost is $COST(A) = 1\$$. Fig. 3 shows the task execution process.

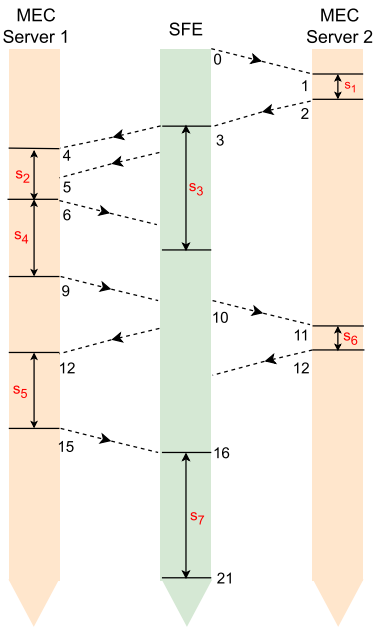


Fig. 3. Process of executing tasks between the SFE and edge servers, including the order of execution, the execution locations, and the corresponding time delays.

IV. PROPOSED ALGORITHM

This section mainly introduces the task offloading method based on EMDRL proposed in this paper, which is used to solve the ITOP in the IIoT-MEC system. This method models the problem as a multi-objective Markov decision process (MOMDP) [15] and adopts EMDRL. It generates a diversified experience set through the population-based method in EA and learns them using the powerful gradient-based method in DRL. In addition, the distillation cross and proximal mutation methods are used to solve the catastrophic forgetting problem caused by the combination of EA and DRL. It improves the learning efficiency and the utilization of learned policies [42]. Next, we will introduce in detail the MOMDP model established and the algorithm proposed in this paper.

A. MOMDP Model

In this paper, the ITOP is a multi-objective optimization problem, so the problem is modeled as a MOMDP. The MOMDP can be represented as a five-tuple $\mathcal{S}$, $\mathcal{A}$, $\mathcal{P}$, r, γ, where $\mathcal{S}$ and $\mathcal{A}$ represent the state space and action space, respectively. $\mathcal{P}$ represents the state transition probability. In this paper, since there are multiple optimization objectives, $r = (r_1, r_2, \ldots, r_n)$ represents the vector reward value, where n represents the number of objectives, and γ represents the reward discount factor. The state space, action space, and reward function are described as follows.

1) State Space:

$$\mathcal{S} = \{s_t | s_t = (t-1, o_t, cp_l, cp_m = (cp_{m_1}, \ldots, cp_{m_f}))\} \quad (16)$$

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:55 UTC from IEEE Xplore. Restrictions apply.

where s_t represents the state of the IIoT-MEC environment at time slot *t*, $t-1$ represents the number of tasks already executed under s_t. And o_t represents the t-th task in the execution order vector *O* under s_t. cp_l represents the computing capacity of the SFE under s_t, and $cp_m = (cp_{m_1}, \ldots, cp_{m_f})$ represents the computing capacity of MEC servers under s_t.

2) Action Space:

$$\mathcal{A} = \{a_t | a_t \in \{0, 1, \ldots, F\}\} \tag{17}$$

where a_t represents the action taken by SFE under s_t. If $a_t = 0$, it means the task is executed on SFE. Otherwise, it means the task is executed on the a_t-th MEC server.

3) Reward Function: The proposed algorithm aims to minimize the delay *D*(*A*), energy consumption *E*(*A*), and cost *COST*(*A*). Therefore, the vector reward function is defined as:

$$r_t = \left(r_t^D, r_t^E, r_t^{COST}\right) \tag{18}$$

where r_t^D, r_t^E, and r_t^{COST} represent the reward values for delay, energy consumption, and cost, respectively. In this paper, to simultaneously minimize these three objectives and achieve the goal of maximizing the reward value, we define $r_t^D = D(A_{t-1}) - D(A_t)$, $r_t^E = -E(A_t)$, and $r_t^{COST} = -COST(A_t)$. Where A_t represents a sub-application consisting of the first *t* tasks in the execution order vector *O*. Therefore, the total reward values obtained in time slot *t*, denoted by $R_t = (R_t^D, R_t^E, R_t^{COST})$, are defined as:

$$R_t^D = \sum_{k=t}^{N} \gamma^{k-t} r_k^D = \sum_{k=t}^{N} \gamma^{k-t} D(A_{k-1}) - D(A_k), \tag{19}$$

$$R_t^E = \sum_{k=t}^{N} \gamma^{k-t} r_k^E = -\sum_{k=t}^{N} \gamma^{k-t} E(A_k) \tag{20}$$

$$R_t^{COST} = \sum_{k=t}^{N} \gamma^{k-t} r_k^{COST} = -\sum_{k=t}^{N} \gamma^{k-t} COST(A_k) \tag{21}$$

where γ represents the reward discount factor, *N* represents the number of tasks in SFE. From the above equation, it can be seen that maximizing the reward value is equivalent to minimizing *D*(*A*), *E*(*A*), and *COST*(*A*).

B. EMDRL-ITOP

Fig. 4 shows the basic framework of EMDRL-ITOP. In EMDRL-ITOP, a population-based EA method is combined with the gradient-based method of DRL to generate a diverse set of experiences and learn from them. From Fig. 4, it can be seen that the EA part mainly consists of two stages, namely the Evaluation stage and the Evolutionary stage. In the Evaluation stage, the initial state is obtained from the IIoT-MEC environment, followed by executing the action to receive the reward and the next state. The experience is then stored in the experience pool, and the fitness value for each policy is calculated. Then, the Evolutionary stage performs selection, crossover, and mutation operations on the policies to generate a new population. In addition, the experience pool stores the experiences of the entire evolutionary population and has good diversity. Next, select samples from the experience pool. The samples are trained using the improved Deep Deterministic Policy Gradient (IDDPG), and the training results are fed back to the population. In this way, EA can use the information learned by DRL gradient descent to improve learning efficiency. Table III shows the main symbols used in this section.

TABLE III
MAIN SYMBOL (2)

Symbol	Definition
pop	Offloading strategy population
Ψ	The Genetic memory pool
f	Fitness value, representing the trade-offs among multiple objectives.
EP	The external Pareto archive is used to store the optimal offloading strategies.
Φ	Experience replay pool
π_i	An offloading strategy
ξ	The number of evaluations
$MaxGen$	Number of iterations
k	The population size of offloading strategies.
p_c,p_m	The crossover/mutation probability
epochs	The number of training epochs
S	The set of offloading strategies excluding the elite strategies.
ω	The sensitivity of action-weight perturbation.

Algorithm 1 shows the detailed process of EMDRL-ITOP to address the industrial task offloading problem in IIoT-MEC. During the initialization phase (Lines 1-5) of Algorithm 1, we apply specific configurations to ITOP. First, we initialize the Actor and Critic networks, where the Actor network generates task offloading strategies, while the Critic network evaluates the generated strategies. Additionally, we initialize the strategy population pop, which represents a diverse set of task offloading policies, with each policy corresponding to a potential task offloading solution. Furthermore, we initialize the experience replay pool Φ and the genetic memory pool Φ. These pools store task execution experiences during the execution of EMDRL-ITOP, including the current state of the IIoT-MEC environment s_t, the executed action a_t, the received reward r_t, and the next state s_{t+1} at the following time slot.

In the Evaluation stage (Lines 8-10), the performance of each offloading strategy is assessed through interactions with the IIoT-MEC environment. Algorithm 2 outlines the process of this stage. Specifically, for each strategy π_i, the agent retrieves the current state s_t from the IIoT-MEC environment, which includes execution progress, task execution order, real-time computational capacity, and other relevant factors (as described in Equation (16)). Based on π_i, the agent selects an action a_t, determining whether the task should be executed on the SFE or an edge server. Subsequently, the agent executes action a_t in state s_t, obtaining the reward r_t and transitioning to the next state s_{t+1}. The reward r_t directly reflects the impact of the current offloading strategy on delay, energy consumption, and overall cost. Furthermore, the experience tuple (s_t, a_t, r_t, s_{t+1}) is stored in both the experience replay pool and the genetic memory pool for future use. To ensure the stability and reliability of the evaluation results, each strategy is assessed ξ times. Finally, the average fitness value f is computed to more accurately reflect the performance of the current strategy within the IIoT-MEC environment.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:55 UTC from IEEE Xplore. Restrictions apply.

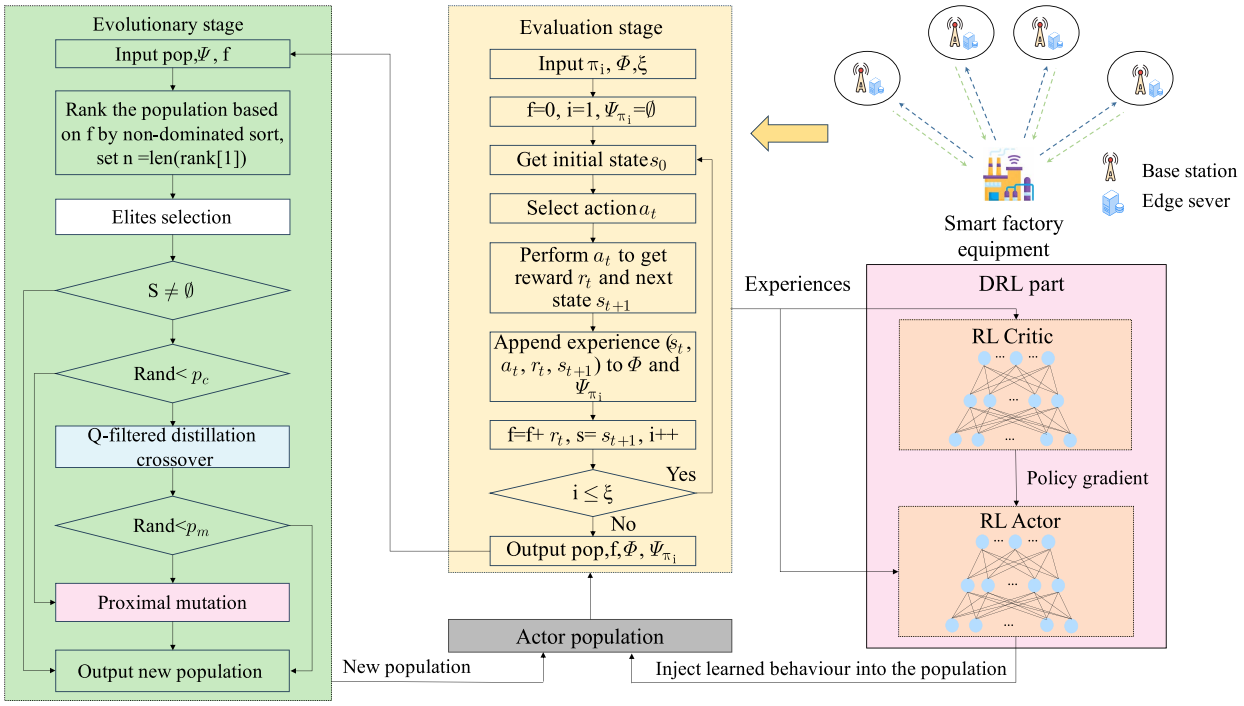


Fig. 4. Basic framework of EMDRL-ITOP. It includes the main components of the algorithm (Evaluation phase, Evolution phase, DRL part, etc.) and the interactions among these components, which illustrate the operational mechanism and workflow of the algorithm.

Algorithm 1 EMDRL-ITOP Algorithm

Require: Number of iterations *MaxGen*, number of evaluations ξ, population size k, crossover probability p_c, mutation probability p_m, number of training epochs *epochs*, external Pareto archive *EP*.
Ensure: External Pareto archive *EP*.
1: Initialize actor π_r and critic Q_r with weights θ^{π} and θ^{Q}, respectively.
2: Initialize target actor π_r' and target critic Q_r' with weights $\theta^{\pi'}$ and $\theta^{Q'}$, respectively.
3: Initialize the policy population *pop* with k policies and initialize the experience replay pool Φ.
4: Initialize a genetic memory pool Ψ_{π_i} of size n_g for each policy $\pi_i \in pop$.
5: Set $g = 1$, generate a random number *rand* in [0, 1], and initialize $EP = \phi$.
6: **for** $g \leq MaxGen$ **do**
7: // Evaluation Stage
8: **for** $\pi_i \in pop$ **do**
9: $pop, f, \Phi, \Psi_{\pi_i} = Evaluation(\pi_i, \Phi, \xi)$ // Algorithm 2
10: **end for**
11: // Evolutionary stage
12: Rank the population using non-dominated sorting based on the fitness values *f*, and record the number of the best individuals $n = len(rank[1])$.
13: Select the n best policies π_i as elites and put them into the external Pareto archive *EP*, then save the remaining policies in a set *S*.
14: **for** $\pi_i \in S$ **do**
15: $pop = DCPM(\pi_i, p_c, p_m, epochs, pop)$ // Algorithm 3
16: Update *EP* using the new population.
17: **end for**
18: _, _, Φ, _ $= Evaluation(\pi_r, \Phi, \xi = 1)$. // Algorithm 2
19: // DRL part
20: Update networks parameters by $IDDPG(n_r, \Phi)$. // Algorithm 4
21: **if** g mod $\mu = 0$ **then**
22: Copy individuals from the DRL to the population and update *EP*.
23: **end if**
24: **end for**

In the Evolutionary stage (Lines 12-18), selection, crossover, and mutation operations are performed on the strategy population to generate a new set of strategies. First, the strategy population is ranked using a non-dominated sorting method [43]. Based on this ranking, an elite selection strategy is applied, where the top n strategies are stored in the EP. These elite strategies represent the best-performing solutions in the current population, achieving a well-balanced trade-off between delay, energy consumption, and cost. Next, the remaining offloading strategies undergo crossover and mutation operations to generate new candidate strategies for the next iteration.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:55 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 Evaluation

Require: Policy π_i, experience replay pool Φ, number of evaluations ξ.
Ensure: Policy population *pop*, fitness value *f*, experience replay pool Φ, genetic memory pool Ψ_{π_i}.
Set $f = 0$.
for $c = 1{:}\xi$ **do**
  Obtain the initial state s_0 from the IIoT-MEC environment.
  for $t = 1{:}N$ **do**
    Select an action a_t based on the current state.
    Execute the action a_t in state s_t and receive the reward r_t and the next state s_{t+1}.
    Store the tuple (s_t, a_t, r_t, s_{t+1}) in Φ and Ψ_{π_i}.
    Update $f = f + r_t$, set $s = s_{t+1}$.
  end for
end for
Calculate the average fitness value f: $f = \frac{f}{\xi}$.

The distillation crossover method is used in the crossover operation. Specifically, distillation crossover refers to the selective crossing of features from parent individuals to create more advantageous offspring, greatly avoiding catastrophic forgetting in the population. The latest half of the genetic memory pool from that of two parents is selected to form the genetic memory pool Ψ_{π_c} of the offspring, and the order of Ψ_{π_c} is shuffled. Further, the weights of one parent are randomly selected to initialize the weights of the offspring. Finally, the Q network is used to train the N_c samples from Ψ_{π_c}, and θ_c is optimized. The proximal mutation method is used in the mutation operation. Proximal mutation refers to making small changes to the policy to generate offspring that are similar to the parent, improving the stability of the policy.

$$\omega = \sqrt{\sum_{k}^{|\mathcal{A}|}\left(\sum_{j}^{N_m}\nabla_{\theta_c}\theta_c\left(s_j\right)k\right)^2} \tag{22}$$

Specifically, introducing a sensitivity ω to the action-weight perturbation, defined as: where $|\mathcal{A}|$ represents the size of the action space, and N_m represents the batch size of selecting the sample state in the mutation operation. The weights of the offspring are updated through $\theta_c \leftarrow \theta_c + \frac{x}{\omega}$, where *x* represents the introduced Gaussian disturbance [44], further improving the stability of policy updates. Finally, the new population is re-evaluated, and *EP* is updated. Algorithm 3 shows the process of distillation crossover and proximal mutation.

In the DRL part (Lines 20-23), the IDDPG is used to train the policy. Algorithm 4 shows the details of the IDDPG. n_r samples are selected from the experience replay pool Φ for training. Where the Actor network is used to generate actions and the Critic network is used to evaluate the value of the current policy. Finally, the sample policy is updated by calculating the loss function using gradient descent. During the training process, the policy updated by DRL are copied to the population every μ iterations, realizing the combination of DRL and EA to improve learning efficiency further.

Algorithm 3 DCPM (Distillation Crossover and Proximal Mutation)

Require: Elite policy π_i, crossover probability p_c, mutation probability p_m, number of training epochs *epochs*, policy population *pop*.
Ensure: New population.
if $rand < p_c$ **then**
  // Distillation crossover
  Select the latest $n_g/2$ genetic memories Ψ_{π_i} and Ψ_{π_j} from two parent policies π_i and $\pi_j (i \neq j)$, and add them to the genetic memory pool Ψ_{π_c} of the offspring policy.
  Shuffle the order of the genetic memory pool Ψ_{π_c}.
  Randomly initialize the weights θ_c of the offspring π_c using one of the parent policies.
  for $e = 1{:}epochs$ **do**
    Randomly select a batch of n_c samples from Ψ_{π_c} and train the Q network to optimize θ_c.
  end for
end if
if $rand < p_m$ **then**
  // Proximal mutation
  Initialize the offspring policy π_c and its weights θ_c with the parent policy π_i and its weights θ_i, respectively.
  Randomly select a batch of n_m samples from Ψ_{π_c}, and calculate the sensitivity ω of the action-weight perturbation on the batch of samples s_m using Equation (22).
  Update the weights of the offspring policy $\theta_c \leftarrow \theta_c + \frac{x}{\omega}$, where x represents the introduced Gaussian perturbation.
end if

Algorithm 4 IDDPG

Require: The batch size of samples n_r, experience replay pool Φ.
Ensure: Policy population *pop*, external Pareto archive *EP*.
Randomly sample a small batch of n_r experiences (s_t, a_t, r_t, s_{t+1}) from Φ.
alculate $y_t = r_t + \gamma Q_r'(s_{t+1}, \pi_r'(s_{t+1}|\theta^{\pi'})|\theta^{Q'})$.
Update Q_r by minimizing the loss function *Loss*, where $Loss = \frac{1}{n_r}\sum_t (y_t - Q_r(s_t, a_t|\theta^Q))^2$.
Update π_r using the sample policy gradient.
Update the target networks: $\theta^{\pi'} \leftarrow \tau\theta^{\pi} + (1-\tau)\theta^{\pi'}$ and $\theta^{Q'} \leftarrow \tau\theta^{Q} + (1-\tau)\theta^{Q'}$.

V. Experimental Results and Analysis

A. Parameter Settings

In this paper, an IIoT-MEC environment is considered to consist of three edge servers and an SFE. Assuming the computing capability of the SFE is $0.6GHz$, the computing capability of the edge servers is randomly assigned within the range of [2, 6]*GHz*. For any channel between the SFE and an edge server, its channel gain is randomly selected from the range of $\{-18, -16, -14, -12, -10, -8\}dB$ [33]. The population size *k* is set to 50, and the maximum number of iterations *MaxGen* of the population is set to 2000. To ensure the stability of the fitness value, we set the evaluation times as $\xi = 5$. The crossover and mutation probabilities are set as $p_c = 0.1$ and $p_m = 0.9$, respectively. In the crossover and mutation process, the sample batch sizes n_c and n_m are set to 128 and 256, respectively.

In the DRL part, the Adam optimizer is used to update the network. The learning rates of the Actor and Critic networks are set to $5e^{-5}$ and $5e^{-4}$, respectively. The reward discount

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:55 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
TEST INSTANCES

Instances	N	ρ
I-1	10	0.4
I-2	10	0.6
I-3	20	0.4
I-4	20	0.6
I-5	30	0.4
I-6	30	0.6

TABLE V
PARAMETER SETTINGS OF EMDRL-ITOP

Parameter	Value
Number of MEC servers (F)	3
Computing capability of SFE (cp_l)	$0.6GHz$
Computing capability of edge server m_f (cp_{m_f})	$[2,6]GHz$
Coefficient associated with chip (h)	10^{-26}
Number of CPU cycles required for task s_i (c_i)	$[0.1,0.5]GHz$
Input data size of task s_i (ids_i)	$[5000,6000]kb$
Output data size of task s_i (ods_i)	$[500,1000]kb$
Transmission power of SFE (p^s)	$0.5Watt$
Receiving power of SFE (p^r)	$0.1Watt$
Noise power (σ^2)	$10^{-6}Watt$
Channel bandwidth (B)	$0.6MHz$
Coefficients of computing cost (α) and β	2.5×10^{-4}
Reward discount factor (γ)	0.99
Number of training epochs ($epochs$)	2000
Population size (k)	50
Maximum number of iterations ($MaxGen$)	2000
Evaluation times (ξ)	5
Crossover probability p_c and mutation probability (p_m)	0.1, 0.9
Capacity of genetic memory pool Ψ (n_g)	800

factor is set as $\gamma = 0.99$, and the number of training rounds is set as $epochs = 2000$. The capacity of the genetic memory pool Ψ is set as $n_g = 400$, and the sample batch size selected from the experience replay pool is set as $n_r = 128$. Every $\mu = 10$ generation, individuals in the DRL part are synchronized to the population. As shown in Table IV, six instances are tested in this paper, with different numbers of tasks N and ρ values. The parameter ρ is used to adjust the topological characteristics of DAG. When $\rho = 0.4$, the dependency among tasks is strong, resulting in lower parallel execution potential. In contrast, when $\rho = 0.6$, the independence between task nodes is enhanced, allowing for higher parallel computing capability. The experimental data obtained in this paper is the average result of 20 runs on each test instance. Table V shows the parameter settings used in this paper.

B. Performance Measures

Three widely used evaluation indicators are adopted to evaluate the performance of EMDRL-ITOP, namely, the Inverted Generational Distance (IGD), the Hypervolume (HV), and the Comprehensive Objective Indicator (COI).

Let PF_{ref} and PF_{know} denote the true Pareto front and the approximated Pareto front, respectively. It is worth noting that in dealing with ITOP in IIoT-MEC, we cannot determine its PF_{ref}. Therefore, a common method is adopted [38], which involves collecting the best solutions found by all algorithms throughout their runs thus far, and obtaining the PF associated with those non-dominated solutions as the PF_{ref}.

1) Inverted Generational Distance: The Inverted Generational Distance can evaluate not only the convergence performance of the algorithm but also the diversity of the Pareto front obtained by the algorithm. It is defined as:

$$IGD = \frac{\sum_{z\in PF_{ref}} d(z, PF_{know})}{|PF_{ref}|} \tag{23}$$

where $d(z, PF_{know})$ represents the Euclidean distance between point $z \in PF_{ref}$ and the nearest point in PF_{know}, and $|PF_{ref}|$ represents the number of points in PF_{ref}. A smaller IGD value indicates a better convergence performance and diversity of the algorithm.

2) Hypervolume: The Hypervolume measures the quality of the solution set. The larger the HV, the better the performance of the algorithm. The HV is defined as:

$$HV = Leb\big(\cup_{y\in NDS}[y, Z_{ref}]\big) \tag{24}$$

where *Leb* represents the Lebesgue measure, *NDS* represents the non-dominated solution set. Z_{ref} represents the reference point set. In this paper, Z_{ref} is set to an all-zero vector.

3) Comprehensive Objective Indicator: Since the ITOP has three objectives, that is, delay, energy consumption, and cost, a Comprehensive Objective Indicator (COI) is designed to evaluate the overall performance of the algorithm. The smaller the COI, the better the performance of the algorithm. The weight generation method proposed in [45] is adopted in this paper, and the weights are stored in $\mathcal{W}$. Assigning a weight to each objective, the three objective vectors are aggregated into COI by weight sum, defined as:

$$COI^j_{\mathbf{w}} = w_1 \cdot D^j_{\mathbf{w}}(A) + w_2 \cdot E^j_{\mathbf{w}}(A) + w_3 \cdot COST^j_{\mathbf{w}}(A) \tag{25}$$

where $\mathbf{w} = (\mathbf{w_1}, \mathbf{w_2}, \mathbf{w_3})$ represents a weight vector in $\mathcal{W}$. Each run of the algorithm generates n_o objective vectors, denoted as $(D^j_{\mathbf{w}}(A), E^j_{\mathbf{w}}(A), COST^j_{\mathbf{w}}(A))$ for $j = 1, \ldots, n_o$. Underweight $\mathbf{w}$, the optimal value for each objective among all objective vectors is defined as:

$$COI^{best}_{\mathbf{w}} = w_1 \cdot D^{best}_{\mathbf{w}}(A) + w_2 \cdot E^{best}_{\mathbf{w}}(A) + w_3 \cdot COST^{best}_{\mathbf{w}}(A) \tag{26}$$

where $best = argmin_{j=1,\ldots,n_o} COI^j_{\mathbf{w}}$. Based on this, the best value for each objective among all objective vectors is calculated, thus the optimal objective vector for each weight vector in $\mathcal{W}$ is obtained. Then, we calculate the Average Delay (AD), Average Energy Consumption (AEC), Average Cost

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:55 UTC from IEEE Xplore. Restrictions apply.

(ACOST), and Average Comprehensive Objective Indicator (ACOI), defined as:

$$AD = \frac{\sum_{\mathbf{w}\in\mathcal{W}} D_{\mathbf{w}}^{best}(A)}{|\mathcal{W}|} \tag{27}$$

$$AEC = \frac{\sum_{\mathbf{w}\in\mathcal{W}} E_{\mathbf{w}}^{best}(A)}{|\mathcal{W}|} \tag{28}$$

$$ACOST = \frac{\sum_{\mathbf{w}\in\mathcal{W}} COST_{\mathbf{w}}^{best}(A)}{|\mathcal{W}|} \tag{29}$$

$$ACOI = \frac{\sum_{\mathbf{w}\in\mathcal{W}} COI_{\mathbf{w}}^{best}(A)}{|\mathcal{W}|} \tag{30}$$

C. Experimental Comparison

To demonstrate the performance of EMDRL-ITOP, it is compared with two advanced EAs and one EDRL algorithm on six instances, as follows:

1) INSGA-II: A fast non-dominated sorting genetic algorithm based on elite selection [25] is used to solve the ITOP. The population size is set to 50, and the crossover and mutation probabilities are set to 0.1 and 0.9, respectively.
2) IMOEA/D: A decomposition-based multi-objective evolutionary algorithm [26] is used to solve the ITOP. The population size is set to 50, and the number of neighbors for each subproblem is set to 10.
3) INSGA-III: An non-dominated sorting genetic algorithm III [22] is adopted to solve the ITOP. The population size is set to 50, and the crossover and mutation probabilities are set to 0.1 and 0.9, respectively.
4) IEDRL: An evolutionary reinforcement learning algorithm that combines evolutionary algorithms and reinforcement learning [35] is used to solve the ITOP in IIoT-MEC. The population size is set to 50, and the mutation probability is set to 0.9.
5) IEMDRL: An improved evolutionary multi-objective reinforcement learning algorithm that combines evolutionary algorithms and multi-objective reinforcement learning [37] is used to solve the computation offloading problem in the UAV-assisted MEC system. The population size is set to 50.
6) EMDRL-ITOP: The proposed algorithm in this paper.

For fairness, INSGA-II, IMOEA/D, INSGA-III, IEDRL, and IEMDRL adopt the same parameter settings as EMDRL-ITOP.

Fig. 5 shows the convergence of IGD values for the four algorithms on six instances as the number of iterations increases. Firstly, it can be observed that the IGD values for INSGA-II, IMOEA/D, and INSGA-III are the largest, indicating that their performance is not ideal. This is because the IIoT-MEC environment is a highly dynamic environment with uncertainty, for example, the number of channels is constantly changing. While traditional algorithms such as INSGA-II, IMOEA/D, and INSGA-III perform well in static environments, they cannot interact with dynamic environments. This means it cannot simultaneously achieve low delay, low energy consumption, and low cost. Therefore, INSGA-II, IMOEA/D, and INSGA-III are difficult to converge in dynamic environments, which is why they perform worse than IEDRL, IEMDRL, and EMDRL-ITOP in solving the ITOP in IIoT-MEC.

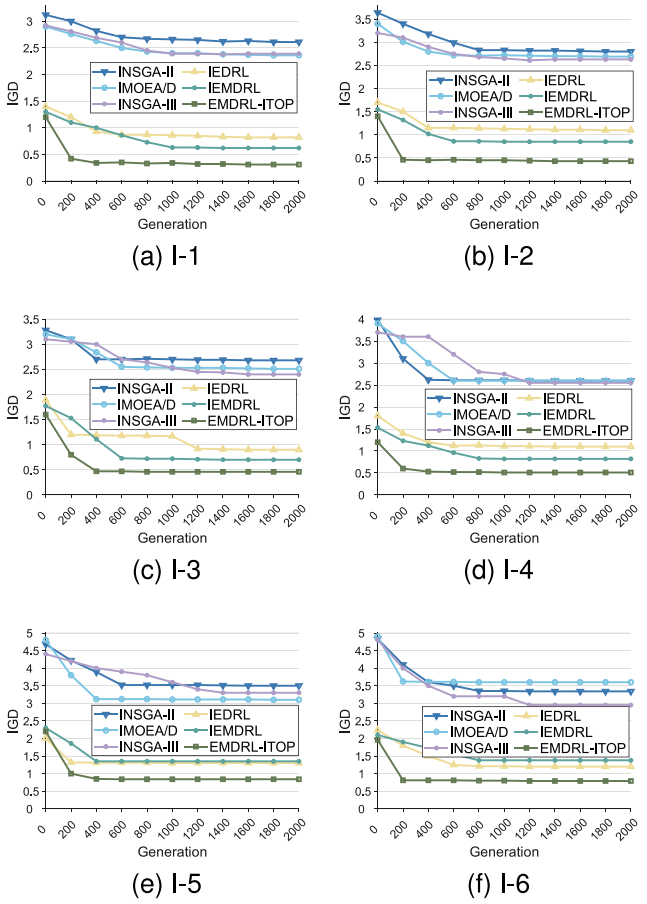


Fig. 5. Convergence of different algorithms based on IGD under six instances.

Secondly, IEDRL combines evolutionary algorithms and DRL, which cannot only adaptively adjust its strategy to environmental changes but also has strong self-learning ability and good robustness. This allows IEDRL to find the optimal policy in a vast search space and solve complex dynamic problems. However, IEDRL only combines EA and DRL simply. The small changes in parameters during the crossover and mutation process may affect the performance of the DRL part, leading to catastrophic forgetting in the evolutionary process. And the inability of offspring to combine the good behavior of their parents. Moreover, when IEDRL deals with multi-objective problems, it transforms multiple objectives into a single objective by weighted sum, which ignores the conflicts between multiple objectives. So IEDRL has certain limitations in handling multi-objective problems. This is why the performance of IEDRL is inferior to EMDRL-ITOP.

In addition, IEMDRL combines the evolutionary algorithm with a multi-objective DRL algorithm, which can well weigh the conflicts between multiple objectives. Therefore, the performance of IEMDRL is better than IERL. It is worth noting that the DRL algorithm of IEMDRL uses an improved Proximal Policy Optimization (PPO), which often achieves good performance when processing continuous action spaces.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:55 UTC from IEEE Xplore. Restrictions apply.

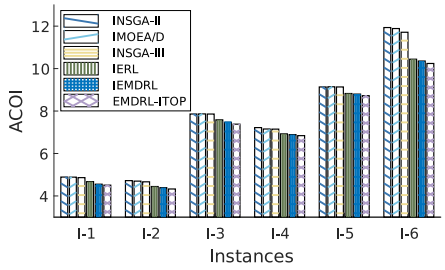


Fig. 6. ACOI values of four algorithms under six instances.

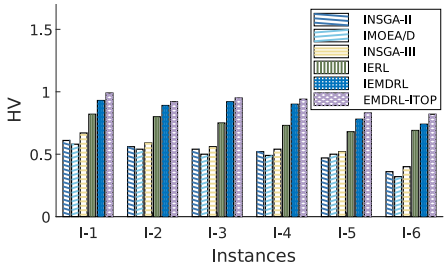


Fig. 7. HV values of four algorithms under six instances.

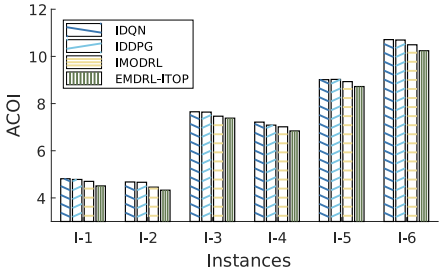


Fig. 8. ACOI values of different DRL algorithms under six instances.

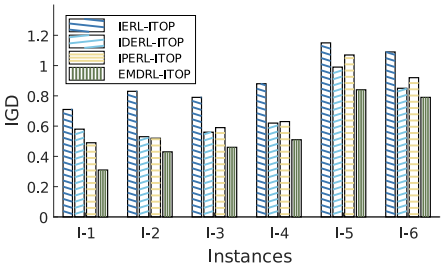


Fig. 9. IGD values of different algorithms under six instances.

However, in the task offloading model of IIoT-MEC, the action space is often discrete, so the PPO algorithm is not applicable. This is why IEMDRL shows poor convergence performance.

Fig. 6 and Fig. 7 show the ACOI values, and the HV values of different algorithms on six instances. It can be observed that EMDRL-ITOP achieves the smallest ACOI values and the biggest HV values in all instances. indicating that its performance is the best, which is also supported by Fig. 5. EMDRL-ITOP uses non-dominated sorting and elite selection mechanism to select and retain the best-performing parents. Additionally, EMDRL-ITOP utilizes distillation crossover and proximal mutation methods, which enable offspring to better combine the best features of their parents and solve the catastrophic forgetting problem in IEDRL. Furthermore, to better handle multi-objective optimization problems, EMDRL-ITOP establishes a MOMDP model and extends the reward function to make each element correspond to an objective, thereby obtaining a set of optimal task offloading decisions.

In addition, EMDRL-ITOP is also compared with two other advanced DRL algorithms and a multi-objective reinforcement learning algorithm, all of which use the same parameter settings as EMDRL-ITOP.

1) IDQN: A value function-based deep Q-network algorithm to solve ITOP is proposed in [29]. Multiple objectives are aggregated into a single objective by weight sums for training optimization.
2) IDDPG: An improved deep deterministic policy gradient algorithm for solving ITOP in the IIoT-MEC environment is proposed in [32]. Similar to IDQN, this algorithm also uses the weighted sum method to handle multiple objectives.
3) IMORL: An improved multi-objective reinforcement learning algorithm proposed in [33] handles ITOP in the IIoT-MEC environment through dynamic weight adjustment.

Fig. 8 shows the ACOI values of the four algorithms under six instances. It can be observed that the values of IDQN and IDDPG are the biggest. This is because ITOP in IIoT-MEC is an MOP, and single-policy DRL algorithms cannot handle multiple objectives simultaneously. They can only assign weights to each objective and use the weighted sum method to formulate a scalarization function for optimization.

However, in the complex IIoT-MEC environment, users' preferences for different objectives may change. Therefore, it is difficult for IDQN and IDDPG algorithms to determine appropriate weights, so their algorithm performance is not ideal. Unlike IDQN and IDDPG, IMORL can dynamically adjust weights to handle ITOP. Optimizing the objectives based on the initial and current weights can prevent the algorithm from overfitting to the current weights. Therefore, its algorithm performance is relatively good in the six instances. However, selecting appropriate initial weights and weight spaces is a challenging problem in IMORL, and its convergence cannot be guaranteed. In contrast, the EMDRL-ITOP algorithm can effectively solve the hyperparameter problem in DRL through evolutionary algorithms, and better balance multiple objectives in ITOP by combining EA and DRL, thus formulating a set of optimal task offloading decisions.

To prove the validity of distillation crossover and proximal mutation in EMDRL-ITOP, the following experiment was conducted:

1) IEDRL-ITOP: The EMDRL-ITOP algorithm without proximal mutation and distillation crossover.
2) IDEDRL-ITOP: The EMDRL-ITOP algorithm without proximal mutation.
3) IPEDRL-ITOP: The EMDRL-ITOP algorithm without distillation crossover.
4) EMDRL-ITOP: The proposed algorithm in this paper.

Fig. 9 shows the IGD values of six instances under different conditions. It can be seen that when distilled crossover and

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:55 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
AD VALUES OF ALL COMPARED ALGORITHMS UNDER SIX INSTANCES (SEC.)

Algorithms	I-1	I-2	I-3	I-4	I-5	I-6
INSGA-II	6.1402	5.7425	10.1237	9.9875	14.7243	13.9823
IMOEA/D	6.1423	5.6982	10.4142	9.9124	14.6852	13.8264
INSGA-III	6.1045	5.6627	10.2369	9.8853	14.6235	13.8665
IDQN	5.9112	5.4759	9.9874	9.8457	14.4548	13.8542
IDDPG	5.8163	5.8674	10.1478	9.9654	14.3574	14.0536
IMORL	5.9643	5.1989	10.2548	9.7956	14.3549	13.6886
IEDRL	5.7815	5.3724	9.9254	9.7955	14.1954	13.6329
IEMDRL	5.7555	5.2158	9.9126	9.7747	14.1753	13.6025
EMDRL-ITOP	**5.7319**	**5.1778**	**9.8885**	**9.7621**	**14.1378**	**13.5987**

TABLE VII
AE VALUES OF ALL COMPARED ALGORITHMS UNDER SIX INSTANCES (J)

Algorithms	I-1	I-2	I-3	I-4	I-5	I-6
INSGA-II	4.8069	4.2548	11.1347	10.9003	14.5704	14.1387
IMOEA/D	4.8013	4.2364	11.0953	10.9114	14.5715	14.1423
INSGA-III	4.7895	4.2256	11.1125	10.9024	14.5668	14.1399
IDQN	4.7388	4.1982	11.1568	10.9067	14.5612	14.1349
IDDPG	4.7235	4.1423	11.1485	10.8936	14.5603	14.1197
IMORL	**4.7001**	4.1169	11.0654	10.8853	14.5596	14.1283
IEDRL	4.7121	4.1858	11.0245	10.8861	14.5578	14.1142
IEMDRL	4.7119	4.1536	11.0125	10.8790	14.5536	14.1139
EMDRL-ITOP	4.7004	**4.1075**	**11.0086**	**10.8751**	**14.5507**	**14.1135**

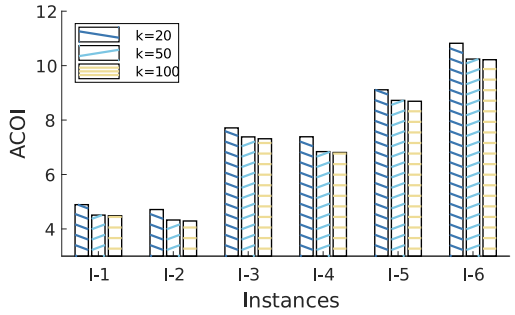


Fig. 10. ACOI values of different population sizes under six instances.

TABLE VIII
TRAINING TIME OF EMDRL-ITOP WITH DIFFERENT POPULATION SIZES UNDER SIX INSTANCES (SEC.)

Population Size	I-1	I-2	I-3	I-4	I-5	I-6
k = 20	887	785	1566	1473	2538	2436
k = 50	1824	1623	3317	3017	4899	4585
k = 100	3454	3122	6367	6159	9249	9013

proximal mutation techniques are not used, the performance of the algorithm is not ideal. This is because traditional crossover and mutation operations have the problem of catastrophic forgetting and poor population diversity. In addition, when only distilled crossover or only proximal mutation technology is used, although the algorithm performance is improved to a certain extent, the best results are not achieved. This is because when only distilled crossover is used, the information of the parent cannot be fully utilized, resulting in the loss of effective resources. When only proximal mutation is used, the diversity of the population cannot be well guaranteed, and it is not easy for the parameters of the offspring to obtain optimal values. It can be seen that the use of distilled crossover and proximal mutation in EMDRL-ITOP is effective.

Furthermore, we analyze the impact of different population sizes on algorithm performance. As shown in Fig. 10, we calculate the ACOI values for population sizes of 20, 50, and 100. Among the six instances, when the population size k is set to 20, the ACOI value is the highest. This is because a smaller population size leads to insufficient diversity, making the algorithm prone to getting stuck in local optima. In contrast, when $k = 100$, the ACOI value is the lowest, indicating that the EMDRL-ITOP can obtain a better offloading strategy. However, as shown in Table VIII, the training time for $k = 100$ is significantly longer, nearly twice that of $k = 50$. Therefore, considering both algorithm performance and computational efficiency, we set the population size to 50 in this paper.

Tables VI-X show the AD, AE, ACOST, and ACOI values of all compared algorithms under six instances, with the optimal solutions highlighted in bold. As shown in Table VI, the EMDRL-ITOP algorithm achieves the smallest AD value in all six instances, indicating that it has the lowest delay among all other algorithms. Table VII shows the AE values of the algorithms in different instances. It can be seen that the EMDRL-ITOP algorithm achieves the best results in most cases (except for I-1), meaning that it has the lowest energy consumption when solving ITOP. Although IMORL achieved the lowest energy consumption in test instance I-1, it also resulted in higher delay and cost. In terms of ACOST, except for instances I-2 and I-3, the EMDRL-ITOP algorithm obtains the lowest cost. Although IEDRL and IDDPG achieved the lowest cost in I-2 and I-3, respectively, they also resulted

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:55 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
ACOST VALUES OF ALL COMPARED ALGORITHMS UNDER SIX INSTANCES ($)

Algorithms	I-1	I-2	I-3	I-4	I-5	I-6
INSGA-II	0.0069	0.0061	0.0146	0.0127	0.0242	0.0241
IMOEA/D	0.0070	0.0057	0.0147	0.0131	0.0235	0.0243
INSGA-III	0.0069	0.0052	0.0145	0.0126	0.0229	0.0236
IDQN	0.0068	0.0046	0.0150	0.0123	0.0210	0.0226
IDDPG	0.0062	0.0047	**0.0128**	0.0118	0.0213	0.0227
IMORL	0.0067	0.0043	0.0135	0.0115	0.0211	0.0221
IEDRL	0.0061	**0.0040**	0.0141	0.0112	0.0196	0.0215
IEMDRL	0.0059	0.0042	0.0136	0.0109	0.0194	0.0214
EMDRL-ITOP	**0.0057**	0.0041	0.0131	**0.0105**	**0.0191**	**0.0213**

TABLE X
ACOI VALUES OF ALL COMPARED ALGORITHMS UNDER SIX INSTANCES

Algorithms	I-1	I-2	I-3	I-4	I-5	I-6
INSGA-II	4.8913	4.7248	7.8653	7.2283	9.1487	11.9383
IMOEA/D	4.8875	4.7045	7.8714	7.1554	9.1454	11.8974
INSGA-III	4.8625	4.6654	7.859	7.148	9.135	11.712
IDQN	4.8136	4.6823	7.6524	7.2124	9.0189	10.7135
IDDPG	4.7847	4.6685	7.6454	7.0914	9.0356	10.7087
IMORL	4.7042	4.4656	7.4786	7.0234	8.9354	10.4953
IEDRL	4.6887	4.4496	7.5922	6.9354	8.8386	10.4452
IEDRL	4.6054	4.4153	7.5236	6.8856	8.7826	10.3048
EMDRL-ITOP	**4.5147**	**4.3345**	**7.3857**	**6.8479**	**8.7268**	**10.2438**

TABLE XI
EXECUTION TIME OF ALL COMPARED ALGORITHMS UNDER SIX INSTANCES (SEC.) (TRAINING TIME OUTSIDE PARENTHESES, INFERENCE TIME INSIDE PARENTHESES)

Algorithms	I-1	I-2	I-3	I-4	I-5	I-6
INSGA-II	541(-)	453(-)	1198(-)	824(-)	1689(-)	1277(-)
IMOEA/D	534(-)	467(-)	1214(-)	822(-)	1702(-)	1268(-)
INSGA-III	587(-)	432(-)	1123(-)	837(-)	1742(-)	1273(-)
IDQN	1477(3.925)	1213(3.725)	2814(7.773)	2436(7.542)	4174(10.821)	3578(10.662)
IDDPG	1566(3.972)	1302(3.756)	2970(7.812)	2668(7.429)	4322(10.663)	3755(10.721)
IMORL	1593(4.114)	1295(3.873)	3012(7.882)	2883(7.616)	4411(11.343)	3964(11.129)
IEDRL	1916(4.290)	1637(3.922)	3387(8.066)	3112(7.844)	4925(11.903)	4523(11.411)
IEMDRL	1811(4.663)	1598(4.314)	3244(8.241)	3087(8.002)	4876(12.120)	4432(11.896)
EMDRL-ITOP	1824(4.323)	1623(3.963)	3317(8.112)	3107(7.884)	4899(11.895)	4585(11.538)

in higher delay and energy consumption. Overall, INSGA-II, IMOEA/D, INSGA-III, IDQN, IDDPG, IMORL, IEDRL, and IEMDRL cannot achieve a good trade-off between the conflicting objectives of delay, energy consumption, and cost when solving ITOP. Although EMDRL-ITOP did not obtain the smallest energy consumption and cost in all instances, its ACOI is the smallest, indicating that its performance is the best. It can achieve a good trade-off between the three objectives, thus satisfying the user's requirements for simultaneous optimization of delay, energy consumption, and cost.

In addition, Table XI compares the training and inference time of all algorithms. Traditional evolutionary algorithms (INSGA-II, IMOEA/D, INSGA-III) cannot perform real-time inference due to the lack of online decision-making mechanisms (marked as "-"). Experimental results show that although EMDRL-ITOP requires longer training time (1623-4899s) than other algorithms, it significantly outperforms both traditional evolutionary algorithms and DRL baselines in solution quality. This computational overhead stems from the synergistic load of algorithm fusion: 1) The dynamic elite selection strategy necessitates maintaining non-dominated solution sets, increasing population iteration costs; 2) The DRL module requires optimizing deep neural network parameters; 3) The distillation crossover and proximal mutation mechanisms demand simultaneous maintenance of population diversity and prevention of catastrophic forgetting. Nevertheless, EMDRL-ITOP achieves a balance between algorithmic performance and computational efficiency, with inference time limited to 4-12 seconds while ensuring solution quality, thereby meeting real-time decision-making requirements in the IIOT-MEC environment.

VI. CONCLUSION AND FUTURE WORKS

In this paper, the ITOP in IIoT-MEC is modeled as a multi-objective optimization problem, and a MOMDP model is established to simultaneously optimize the three conflicting objectives of delay, energy consumption, and cost. Considering the high complexity and uncertainty of the IIoT-MEC environment, we propose EMDRL-ITOP to solve ITOP. This evolutionary multi-objective DRL algorithm integrates dynamic elite selection, distillation crossover,

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:55 UTC from IEEE Xplore. Restrictions apply.

and proximal mutation mechanisms. Experiments across six instances demonstrate its effectiveness in balancing conflicting objectives in the IIoT-MEC environment. In future work, a cloud-edge collaborative paradigm will be considered to address the ITOP in IIoT-MEC more efficiently.

REFERENCES

[1] O. Peter, A. Pradhan, and C. Mbohwa, "Industrial Internet of Things (IIOT): Opportunities, challenges, and requirements in manufacturing businesses in emerging economies," *Procedia Comput. Sci.*, vol. 217, pp. 856–865, Jan. 2023.

[2] Z. A. E. Houda, B. Brik, A. Ksentini, L. Khoukhi, and M. Guizani, "When federated learning meets game theory: A cooperative framework to secure IIOT applications on edge computing," *IEEE Trans. Ind. Informat.*, vol. 18, no. 11, pp. 7988–7997, Nov. 2022.

[3] M. Satyanarayanan, P. Bahl, R. Caceres, and N. Davies, "The case for VM-based cloudlets in mobile computing," *IEEE Pervasive Comput.*, vol. 8, no. 4, pp. 14–23, Oct.–Dec. 2009.

[4] X. Liu, Z.-Y. Chai, Y.-L. Li, Y.-Y. Cheng, and Y. Zeng, "Multi-objective deep reinforcement learning for computation offloading in UAV-assisted multi-access edge computing," *Inf. Sci.*, vol. 642, Sep. 2023, Art. no. 119154.

[5] J. Cai, H. Fu, and Y. Liu, "Multitask multiobjective deep reinforcement learning-based computation offloading method for Industrial Internet of Things," *IEEE Internet Things J.*, vol. 10, no. 2, pp. 1848–1859, Jan. 2023.

[6] Y. Zhang, H. Huang, L.-X. Yang, Y. Xiang, and M. Li, "Serious challenges and potential solutions for the Industrial Internet of Things with edge intelligence," *IEEE Netw.*, vol. 33, no. 5, pp. 41–45, Sep./Oct. 2019.

[7] L. Lin, W. Zhou, Z. Yang, and J. Liu, "Deep reinforcement learning-based task scheduling and resource allocation for NOMA-MEC in Industrial Internet of Things," *Peer-Peer Netw. Appl.*, vol. 16, pp. 170–188, Jan. 2022.

[8] S. Pan, Z. Zhang, Z. Zhang, and D. Zeng, "Dependency-aware computation offloading in mobile edge computing: A reinforcement learning approach," *IEEE Access*, vol. 7, pp. 134742–134753, 2019.

[9] J. Xu, B. Ai, L. Chen, Y. Cui, and N. Wang, "Deep reinforcement learning for computation and communication resource allocation in Multiaccess MEC assisted railway IoT networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 12, pp. 23797–23808, Dec. 2022.

[10] J. Chen, H. Xing, Z. Xiao, L. Xu, and T. Tao, "A DRL agent for jointly Optimizing computation offloading and resource allocation in MEC," *IEEE Internet Things J.*, vol. 8, no. 24, pp. 17508–17524, Dec. 2021.

[11] L. Zhang et al., "Task offloading and trajectory control for UAV-assisted mobile edge computing using deep reinforcement learning," *IEEE Access*, vol. 9, pp. 53708–53719, 2021.

[12] P. Porambage, J. Okwuibe, M. Liyanage, M. Ylianttila, and T. Taleb, "Survey on multi-access edge computing for Internet of Things realization," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 2961–2991, 4th Quart., 2018.

[13] M. Sun, X. Xu, X. Qin, and P. Zhang, "AoI-energy-aware UAV-assisted data collection for IoT networks: A deep reinforcement learning method," *IEEE Internet Things J.*, vol. 8, no. 24, pp. 17275–17289, Dec. 2021.

[14] Y. Wu, B. Shi, L. P. Qian, F. Hou, J. Cai, and X. S. Shen, "Energy-efficient multi-task multi-access computation offloading via NOMA transmission for IoTs," *IEEE Trans. Ind. Informat.*, vol. 16, no. 7, pp. 481–4822, Jun. 2020.

[15] J. Xu, Y. Tian, P. Ma, D. Rus, S. Sueda, and W. Matusik, "Prediction-guided multi-objective reinforcement learning for continuous robot control," in *Proc. 37th Int. Conf. Mach. Learn.*, Jul. 2020, pp. 10607–10616.

[16] S. Bebortta, D. Senapati, C. R. Panigrahi, and B. Pati, "Adaptive performance modeling framework for QoS-aware offloading in MEC-based IIOT systems," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 10162–10171, Jun. 2022.

[17] Y. Dai, L. Zhao, and L. Lyu, "MEC enabled cooperative sensing and resource allocation for industrial IoT systems," *China Commun.*, vol. 19, no. 7, pp. 214–225, Jul. 2022.

[18] J. Xue, Z. Ye, H. Zhang, and Y. Zhu, "Flexible design of low-delay MEC-VLC integrating network based on Attocell overlap for IIOT," *Electronics*, vol. 11, no. 6, p. 924, 2022.

[19] C.-W. Hsu, Y.-L. Hsu, and H.-Y. Wei, "Energy-efficient and reliable MEC offloading for heterogeneous industrial IoT networks," in *Proc. Eur. Conf. Netw. Commun. (EuCNC)*, 2019, pp. 384–388.

[20] Z. Wu, B. Li, Z. Fei, Z. Zheng, B. Li, and Z. Han, "Energy-efficient robust computation offloading for fog-IoT systems," *IEEE Trans. Veh. Technol.*, vol. 69, no. 4, pp. 4417–4425, Apr. 2020.

[21] X. Jian, L. Wu, K. Yu, M. Aloqaily, and J. Ben-Othman, "Energy-efficient user association with load-balancing for cooperative IIOT network within B5G era," *J. Netw. Comput. Appl.*, vol. 189, Sep. 2021, Art. no. 103110.

[22] K. Peng, H. Huang, B. Zhao, A. Jolfaei, X. Xu, and M. Bilal, "Intelligent computation offloading and resource allocation in IIoT with end-Edge-Cloud computing using NSGA-III," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 3032–3046, Sep./Oct. 2023.

[23] X. Zhang et al., "Optimizing energy-latency tradeoff for computation offloading in SDIN-enabled MEC-based IIOT," *KSII Trans. Internet Inf. Syst.*, vol. 16, no. 12, pp. 4081–4098, Dec. 2022.

[24] Z. Jin, C. Zhang, Y. Jin, L. Zhang, and J. Su, "A resource allocation scheme for joint optimizing energy consumption and delay in collaborative edge computing-Based industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 18, no. 9, pp. 6236–6243, Sep. 2022.

[25] C. Zhou, M. Zhang, Q. Gao, and T. Jing, "A dependency-aware task offloading strategy in Mobile edge computing based on improved NSGA-II," in *Proc. Int. Conf. Wireless Algorithms, Syst., Appl.*, 2022, pp. 638–647.

[26] Z.-Y. Chai, X. Liu, and Y.-L. Li, "A computation offloading algorithm based on multi-objective evolutionary optimization in mobile edge computing," *Eng. Appl. Artif. Intell.*, vol. 121, May 2023, Art. no. 105966.

[27] C. Peng, X. Huang, Y. Wu, and J. Kang, "Constrained multi-objective optimization for UAV-enabled mobile edge computing: Offloading optimization and path planning," *IEEE Wireless Commun. Lett.*, vol. 11, no. 4, pp. 861–865, Apr. 2022.

[28] T. Qiu, J. Chi, X. Zhou, Z. Ning, M. Atiquzzaman, and D. O. Wu, "Edge computing in Industrial Internet of Things: Architecture, advances and challenges," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 4, pp. 2462–2488, 4th Quart., 2020.

[29] X. Wang, Y. Zhang, R. Shen, Y. Xu, and F.-C. Zheng, "DRL-based energy-efficient resource allocation frameworks for uplink NOMA systems," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7279–7294, Aug. 2020.

[30] X. Chen, T. Chen, Z. Zhao, H. Zhang, M. Bennis, and J. Yusheng, "Resource awareness in unmanned aerial vehicle-assisted mobile-edge computing systems," in *Proc. IEEE 91st Veh. Technol. Conf. (VTC)*, 2020, pp. 1–6.

[31] W. Chen, B. Zhu, K. Chi, and S. Zhang, "DRL based offloading of industrial IoT applications in wireless powered mobile edge computing," *IET Commun.*, vol. 16, no. 9, pp. 951–962, 2022.

[32] T. Zhao, L. He, X. Huang, and F. Li, "DRL-based secure video offloading in MEC-enabled IoT networks," *IEEE Internet Things J.*, vol. 9, no. 19, pp. 18710–18724, Oct. 2022.

[33] F. Song, H. Xing, X. Wang, S. Luo, P. Dai, and K. Li, "Offloading dependent tasks in multi-access edge computing: A multi-objective reinforcement learning approach," *Future Gener. Comput. Syst.*, vol. 128, pp. 333–348, Mar. 2022.

[34] M. Yi, P. Yang, M. Chen, and N. T. Loc, "A DRL-driven intelligent joint optimization strategy for computation offloading and resource allocation in ubiquitous edge IoT systems," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 7, no. 1, pp. 39–54, Feb. 2023.

[35] S. Khadka and K. Tumer, "Evolutionary reinforcement learning," 2018, *arXiv:1805.07917*.

[36] A. Pourchot and O. Sigaud, "CEM-RL: Combining evolutionary and gradient-based methods for policy search," 2019, *arXiv:1810.01222*.

[37] F. Song et al., "Evolutionary multi-Objective reinforcement learning based trajectory control and task offloading in UAV-assisted mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 12, pp. 7387–7405, Dec. 2023.

[38] F. Song, H. Xing, S. Luo, D. Zhan, P. Dai, and R. Qu, "A multiobjective computation offloading algorithm for mobile-edge computing," *IEEE Internet Things J.*, vol. 7, no. 9, pp. 8780–8799, Sep. 2020.

[39] H. Lin, L. Yang, H. Guo, and J. Cao, "Decentralized task offloading in edge computing: An Offline-to-online reinforcement learning approach," *IEEE Trans. Comput.*, vol. 73, no. 6, pp. 1603–1615, Jun. 2024.

[40] S. Liu et al., "Dependent task scheduling and offloading for Minimizing deadline violation ratio in mobile edge computing networks," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 2, pp. 538–554, Feb. 2023.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:55 UTC from IEEE Xplore. Restrictions apply.

[41] C. Wang, X. Yu, L. Xu, Z. Wang, and W. Wang, "Multimodal semantic communication accelerated bidirectional caching for 6G MEC," *Future Gener. Comput. Syst.*, vol. 140, pp. 225–237, Mar. 2023.
[42] C. Bodnar, B. Day, and P. Lió, "Proximal distilled evolutionary reinforcement learning," in *Proc. AAAI Conf. Artif. Intell.*, Apr. 2020, pp. 3283–3290.
[43] K. Deb, S. Agrawal, A. Pratap, and T. Meyarivan, "A fast elitist non-dominated sorting genetic algorithm for multi-objective optimization: NSGA-II," in *Proc. Int. Conf. Parallel Problem Solving Nature (PPSN)*, 2000, pp. 849–858.
[44] J. Lehman, J. Chen, J. Clune, and K. O. Stanley, "Safe mutations for deep and recurrent neural networks through output gradients," in *Proc. Genetic Evol. Comput. Conf.*, 2018, pp. 117–124.
[45] Q. Zhang and H. Li, "MOEA/D: A multiobjective evolutionary algorithm based on decomposition," *IEEE Trans. Evol. Comput.*, vol. 11, no. 6, pp. 712–731, Dec. 2007.



Xu Liu received the M.Eng. degree from Tiangong University, Tianjin, China, in 2024. He is currently pursuing the Ph.D. degree with the College of Computer Science, Nankai University. His main research interests include edge computing, the Internet of Things, and trusted execution environments.



Zheng-Yi Chai received the Ph.D. degree in computer science from Xidian University, Xi'an, China, in 2012. He is currently a Professor with the School of Computer Science and Technology, Tiangong University, Tianjin, China. He has published his own academic monograph and over 40 papers in international journals and conferences. He has contributed to the development of 20 copyrighted software systems and invented over ten patents. His research interests include the Internet of Things and multiobjective optimization. He is also a member of the China Computer Federation and a member of ACM and the Chinese Institute of Electronics. He serves as the general chair, the PC chair, and the workshop chair or a TPC member of a number of conferences.



Yan-Yang Cheng is currently pursuing the Ph.D. degree in software engineering with the Computer College, Beijing University of Posts and Telecommunications, China. Her research interest is evolutionary algorithms and its application.



Ya-Lun Li is currently pursuing the Ph.D. degree with Tiangong University, where she is a Lecture. She has published more than 20 papers in some international journals. Her research interests include the Internet of Things and bio-inspired optimization.



Tao Li received the Ph.D. degree in computer science from Nankai University, China, in 2007, where he works as a Professor with the College of Computer Science. His main research interests include heterogeneous computing, machine learning, and intelligent IoT. He is a member of the IEEE Computer Society and the ACM, and a Distinguished Member of CCF.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:55 UTC from IEEE Xplore. Restrictions apply.